

Optimization of Parallel Number Theoretic Transform Algorithms for Multi-Core Digital Signal Processors

Journal:	Journal of Universal Computer Science (J.UCS), Vol. 32, No. 9 (2026), pp. 1354–1374
Authors:	Dingxing Xie, Xinchao Hu, Lian Peng, Guokai Liu, Ning Cheng, Jing Liu, Yang Zhang
PDF Link:	https://lib.jucs.org/article/207845/download/pdf/
Core Domain:	Parallel Computing, Multi-Core Acceleration, CUDA Vector Optimization, Memory Pipeline
HAT-RAG Mapping:	CUDA Similarity Acceleration Engine (cuda_utils.py) & Batch Cosine SIMD Vectorization

1. Executive Summary & Core Research Problem

This research paper from J.UCS introduces a high-performance parallel optimization framework addressing memory access bottlenecks and hardware SIMD stalls in vector transformations. By proposing a four-step matrix decomposition, VLIW/SIMD microkernels, vectorized modular arithmetic across 16 parallel streams, and DMA double-buffering, the paper achieves a **2.84x execution speedup** and a **42.5% reduction in memory bandwidth consumption**.

2. Key Technical Innovations

- Four-Step 2D Matrix Decomposition:** Replaces standard six-step algorithms by restructuring 1D vector arrays into 2D matrices.
- VLIW / SIMD Microkernel:** Applies aggressive loop unrolling and explicit register reuse to saturate functional ALUs.
- 16-Stream Parallel Vector Execution:** Enables multi-threaded SIMD data-paths for modular vector mathematics.
- Double-Buffering DMA Pipelining:** Overlaps memory transfers with active computation.

3. Architectural Mapping to HAT-RAG CUDA Repository

J.UCS Concept	HAT-RAG-CUDA Module	Architectural Synergy & Impact
Four-Step 2D Decomposition	src/cuda_utils.py Batch Cosine Similarity	Transforms 1D chunk comparisons into 2D GPU tensor batch matrix multiplications $Q * D^T$.
16-Way SIMD Stream Parallelism	src/cuda_utils.py PyTorch CUDA Streams	Executes multi-threaded tensor similarity operations across GPU CUDA warps with FP16/BF16 math.
Double-Buffering DMA Pipelining	src/retriever.py Top-Down Logarithmic Traversal	Asynchronously fetches child cluster metadata while scoring parent node embeddings.
Index Locality & Memory Placement	src/hierarchical_tree.py Node Metadata Index	Maintains structured node ID indexing to optimize memory access during top-down branch traversal.

4. Benchmark Results & Research Synergy

Quantitative Benchmark: Latency dropped from 4.82ms down to 1.70ms (2.84x speedup) and bus contention decreased by 42.5%.

Synergy with HAT-RAG: Demonstrates that hardware-aligned parallel algorithms combined with structured memory pipelining deliver scalable throughput. These empirical findings substantiate the CUDA GPU similarity acceleration and logarithmic search engine in HAT-RAG.